



Chapter S2A: Programming Elements

CONTENT

No.	Topic	Syllabus Guide
1	Coding standards & Good Programming Style	2.1.1 – 2.1.3
2	Arrays	2.2.1
	2.1 One dimensional array	
	2.2 Two dimensional array	
	2.3 Assigning a group of elements into an array	
3	Trace table	2.2.6
4	Data validation and verification	2.4.1 - 2.4.2
	4.1 Data verification	
	4.2 Data validation	
5	Program testing	2.4.3 - 2.4.4
	5.1 Syntax errors	
	5.2 Runtime errors	
	5.3 Logic errors	
	5.4 Designing test cases	
	5.5 Designing test plan	
6	Recursion	2.2.5 – 2.2.7
	6.1 Comparing recursion and iteration	
	6.2 Use of call stack	
7	Video References	

1. CODING STANDARDS & GOOD PROGRAMMING STYLE

Coding standards are a set of rules and guidelines established by a software development team to ensure that code is well-structured, maintainable, and readable. Adhering to these standards fosters consistency across a project, making collaboration easier and enhancing code quality. Programming style, influenced by coding standards, company conventions, and individual preferences, plays a crucial role in writing clear and elegant code.

1.1 Principles of Good Programming Style:

1. Use Comments to Enhance Clarity
 - Include essential details such as the programmer's name, date, program description, and version control information.
 - Provide clear explanations of helper functions and algorithms, including their objectives, expected inputs, and outputs. Avoid unnecessary explanations of syntax.
2. Use Meaningful Identifiers
 - Choose descriptive and self-explanatory variable and function names.
 - Avoid single-character identifiers unless widely recognised (e.g., 'i' in loops).
3. Apply Proper Indentation and White Space
 - Format code consistently to improve readability.
 - Use blank lines and indentation to separate logical sections and enhance structure.
4. Adhere to Specifications
 - Ensure that the program meets the given requirements and functions as intended.
5. Avoid Deep Nesting
 - Limit excessive nesting of loops and conditional statements, as deep nesting increases execution time and reduces code readability.

1.2 Coding Standards in A-Level Practical Examinations

In the A-level practical examination, **up to six marks** are awarded for good programming style. Candidates can earn up to two marks for each of the following coding standards:

- Proper use of indentation and white space
- Use of meaningful identifiers
- Inclusion of comments to clarify code

2. ARRAYS

An array is a static data structure made up of a finite, ordered set of elements of the same type, such as integer, real or char. It has a fixed size determined at the time of declaration, meaning the number of elements cannot be changed during execution.

- Finite means that there is a specific number of elements in an array.
- Ordered means that each element has a numbered position in the array, which are called the array index.
- Elements of an array are all stored at contiguous (i.e. adjacent, next to one another) memory locations.
- Access to elements is done using the same name with an index in square brackets.
- Indexing in Python is zero-based (i.e. the first item has an index of 0), while some other programming languages use one-based indexing.

2.1 One-dimensional (1D) Array

In *pseudocode*, a 1D array is declared as follows:

```
DECLARE <identifier> : ARRAY[<L>:<U>] OF <data type>
```

where L and U represent the lower and upper bound respectively.

Alternatively:

```
DECLARE <identifier> : ARRAY[<size>] OF <data type>
```

Example: DECLARE StudentID : ARRAY[1:10] OF INTEGER

	StudentID									
Address	1000	1004	1008	1012	1016	1020	1024	1028	1032	1036
Index	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]	[10]
Content	19	120	82	32	103	98	48	75	72	27

2.2 Two-dimensional (2D) Array

In *pseudocode*, a 2D array is declared as follows:

```
DECLARE <identifier> : ARRAY[<L1>:<U1>,<L2>:<U2>] OF <data type>
```

where L1, L2, U1 and U2 represent the lower and upper bounds of each dimension.

Example: DECLARE Attendance : ARRAY[1:5,1:4] OF CHAR

	Attendance			
Row \ Column	1	2	3	4
Address	1001	1002	1003	1004
1	[1, 1]	[1, 2]	[1, 3]	[1, 4]
Address	1005	1006	1007	1008
2	[2, 1]	[2, 2]	[2, 3]	[2, 4]
Address	1009	1010	1011	1012
3	[3, 1]	[3, 2]	[3, 3]	[3, 4]
Address	1013	1014	1015	1016
4	[4, 1]	[4, 2]	[4, 3]	[4, 4]
Address	1017	1018	1019	1020
5	[5, 1]	[5, 2]	[5, 3]	[5, 4]

	Attendance (with values)			
Row \ Column	1	2	3	4
1	Y	Y	Y	Y
2	Y	N	N	Y
3	N	Y	Y	N
4	Y	Y	Y	Y
5	N	N	N	Y
For example, the following expressions represent specific elements: Attendance[3, 2] = 'Y' Attendance[5, 3] = 'N'				

2.3 Assigning a group of elements into an array

For instance, defining an array to store 30 integer values (e.g. test scores for 30 students) and initialising each element to -1:

```
DECLARE SpellingTest : ARRAY[1:30] of INTEGER
FOR index = 1 TO 30
    SpellingTest[index] ← -1
ENDFOR
```

3. TRACE TABLE

No matter how careful a programmer is, even the simplest programs are likely to contain bugs or errors. One method of debugging is to perform a **dry run**, which involves *manually stepping through the code on paper to observe its behaviour before execution*.

A trace table is a structured method of recording the result of each step during a dry run. It consists of columns where each column represents a variable, condition, or output. However, not every variable, condition, or output in an algorithm requires a column.

The purpose of a trace table is to simulate how a computer would execute an algorithm line by line. By tracking variable changes, condition evaluations, and outputs, programmers can identify logical errors and verify correctness.

Example 1: Simple Loop

The pseudocode contains one variable (`num`), one condition (`num < 100`) and one output (`OUTPUT num`). Perform a dry run for an initial user input of 4.

```
num ← USERINPUT
WHILE num < 100 DO
    num ← num * 2
ENDWHILE
OUTPUT num
```

num	num < 100	OUTPUT
4	True	
8	True	
16	True	
32	True	
64	True	
128	False	128

Example 2: Nested Loops

The pseudocode includes nested loops. The upper limit of the FOR-loop is inclusive. In this example, `i` ranges from 1 to 3 and `j` also ranges from 1 to 3.

```
num ← 0
total ← 0
FOR i ← 1 TO 3
    FOR j ← 1 TO 3
        num ← j * i
        total ← total + num
    ENDFOR
ENDFOR
OUTPUT total
```

i	j	num	total	OUTPUT
		0	0	
1	1	1	1	
	2	2	3	
	3	3	6	
2	1	2	8	
	2	4	12	
	3	6	18	
3	1	3	21	
	2	6	27	
	3	9	36	
				36

Trace tables may contain empty cells. If a variable's value does not change in a particular step, it does not need to be recorded again. This practice helps maintain clarity while analysing algorithm behaviour.

4. DATA VALIDATION and VERIFICATION

4.1 Data Verification

Verification of data refers to confirming that the data entered is correct and consistent. It involves checks to ensure that the data is accurately copied from the source to the destination, with no alteration during the process. If differences are detected, it indicates an error in the transfer.

Verification can take place both when data is entered at the human-machine interface and when it is copied between components within a computerized system. However, it does not guarantee data accuracy, as incorrect data could have been entered initially.

Verification can help detect **transcription** or **transposition** errors – errors that occur during the transfer of data.

A **transcription error** is a specific data entry error that occurs when information is misinterpreted or incorrectly transferred, often due to human operators or optical character recognition (OCR) programs. OCR errors may involve misreading printed characters, particularly when the paper is damaged or the font is unusual, leading to inaccuracies in the data.

Transposition errors occur when two adjacent characters are accidentally switched or reversed during data entry, typically due to typing speed or inattention. This results in incorrect character placement, leading to errors in the data.

Verification techniques could include double-entry verification and visual checks:

Verification technique	Description	Example
Double entry	Entering the data twice and comparing the two entries to ensure consistency.	Re-entering password during the account registration process. 
Visual check	Having a person review the data entered to compare it with the original document.	After data is being entered and displayed on the screen, the user checks and confirms if it is correct.

4.2 Data Validation

Validation is an automated process used to ensure that the input data meets a specified set of rules, ensuring it is sensible, complete, reasonable, and within acceptable boundaries. It helps identify inputs that are obviously incorrect, but it does not verify the accuracy of the data. Data is not processed until it passes the validation checks.

For example, a school information management system can be programmed to only accept a student's date of birth within a certain range of years. However, this does not guarantee that the date of birth is entered correctly, as the program will accept any year within the specified range.

Validation checks may include:

Validation check	Description	Example
Range check	Ensures that the data falls within a predefined range.	The month of the year must be between 1 and 12. Entries outside this range (less than 1 or greater than 12) would be rejected.
Format check	Ensures that the data follows a specific pattern or format.	The first and last characters of a National Registration Identity Card (NRIC) are alphabets, and the digits in between follow a specific structure (e.g., T0812345A).
Length check	Ensures that the data has the correct number of characters.	Every local postal code consists of six digits (e.g., 575955).
Presence check	Ensures that all necessary fields are filled out.	The Name field in an order form cannot be left blank.
Check digit	Uses a check digit to allow the data to be self-checking.	A check digit is an extra digit added to the end of data. The value is determined by a specified algorithm that uses the digits' values and weights. For example, the last digit of Singapore's NRIC.
Type check	Ensures that the data is of the correct data type.	The Name field will be saved as a string, or mathematical calculations will only be performed on numeric types (integer or float).

Example of Check Digit: Validation of Singapore NRIC/FIN

In Singapore, NRIC (National Registration Identity Card) numbers and FIN (Foreign Identification Number) numbers are used to identify Singapore residents and foreigners. NRIC starts with S or T. FIN starts with F, G or M. The number is exactly 9 characters long. The first and the last characters are uppercase letters and the remaining seven are digits, e.g. S9537123Z. To validate the NRIC or FIN number, the following algorithm is used (assuming the 7 digits are in the form ABCDEF):

1. $SUM = (2 \times A) + (7 \times B) + (6 \times C) + (5 \times D) + (4 \times E) + (3 \times F) + (2 \times G)$
2. Add 4 to SUM if NRIC number starts with T or G. (Add 3 if it starts with M.)
3. Find the remainder (RD) when the SUM is divided by 11.
4. Subtract remainder (RD) from 11 to get the difference (DIFF).
5. Convert the difference (DIFF) into a letter using the following table:

DIFF value	1	2	3	4	5	6	7	8	9	10	11
Letter (NRIC)	A	B	C	D	E	F	G	H	I	Z	J
Letter (FIN)	K	L	M	N	P	Q	R	T	U	W	X

Example: NRIC S9537123Z

NRIC	S	9	5	3	7	1	2	3	Z
Digit (D)		9	5	3	7	1	2	3	
Weight (W)		2	7	6	5	4	3	2	
Product (D x W)		18	35	18	35	4	6	6	
SUM(Product)	$18+35+18+35+4+6+6 = 122$								
RD	$122 \% 11 = 1$								
DIFF	$11 - 1 = 10$								
Validation code (read from table)	Z								

The validation code confirms that the NRIC number is valid.

Example of Check Digit: Modulus-11 Arithmetic (You need to memorise this algorithm)

The modulus-11 method to calculate a check digit follows this algorithm:

1. Starting from the units column, assign weights to each digit. The units column gets the weight 2, the tens column gets the weight 3, the hundreds column gets the weight 4 and so on.
2. Multiply each digit by its corresponding weight.
3. Add the individual products.
4. Divide the total sum of products by 11 and note the remainder.
5. Subtract the remainder from 11.

The result is the check digit. If the result is 10, the check digit is represented by the character 'X'. Otherwise, it is an integer.

Example: Product code 1234 (with the fifth digit as the check digit)

Product code (D)	1	2	3	4
Weight (W)	5	4	3	2
Product (D*W)	5	8	9	8
Sum of Product	$5+8+9+8 = 30$			
Remainder after dividing by 11	$30 \% 11 = 8$			
Subtract remainder from 11	$11 - 8 = 3$			
Check digit	3			

Therefore the complete product code is **12343**.

5. PROGRAM TESTING

Programs must run correctly or they are of little value. The careful planning and testing of a program is essential, as is writing maintainable code which assists future updating. When developing programs three main classes of error may occur – syntax, runtime and logic error.

5.1 Syntax error

This occurs when the code does not follow the grammar of the programming language, which means the rules of the language have been broken. It is detected when the compiler or interpreter try to translate the source code to machine code. Examples include:

- misspelling a statement, e.g. writing pint instead of print, and
- unmatched parenthesis, e.g. opening a bracket, but not closing it.

A program cannot run if it has syntax errors, which must be fixed before execution. Most syntax errors are spotted by the programmer during coding, but if overlooked, they are identified during the syntax analysis stage of the program's translation. A good Integrated Development Environment (IDE) usually points out any syntax errors to the programmer during the coding.

5.2 Runtime error

Runtime errors are errors detected during program execution. These errors can occur if a mistake is made in the processing algorithm or as a result of external effects not catered for by the program, such as lack of memory or unexpected data input.

Types of runtime errors	Description	Examples
Input Error	Occurs when the program receives incorrect data from the user.	User entering a text when a number is expected, or enters a date in an incorrect format.
Index Out of Bounds	Happens when trying to access an invalid element in an array.	Trying to access the 6th element in an array with only 5 items.
Undeclared Variable	Occurs when trying to use a variable that has not been declared.	Trying to use a variable x without declaring it first.
Arithmetic Error	Happens during incorrect arithmetic operations.	Dividing by zero, or attempting to store a number larger than a variable can hold.
Out of Memory	Occurs when the program uses more memory than is available.	Trying to create a large array that exceeds the computer's available memory.
Stack Overflow	Occurs when there is excessive nesting or recursion.	Recursively calling a function without a base case, causing the program to crash.

Infinite Loop	Occurs when a loop never ends due to missing exit condition.	A while loop that never reaches its exit condition, freezing the program.
Memory Leak	Happens when memory is allocated but not properly released.	Allocating memory in a program but never freeing it, causing a gradual decrease in available memory.
Type Mismatch Error	Occurs when a variable is used with an incompatible type in an operation.	Trying to add a string and an integer, or passing the wrong data type to a function.
File I/O Error	Happens when there are issues with file operations.	Trying to read a file that doesn't exist or attempting to write to a read-only file.
Overflow	Occurs when a value exceeds the maximum limit of a data type.	Trying to store a number larger than the variable can hold, such as 256 in an 8-bit unsigned integer.
Underflow	Happens when a value goes below the minimum limit of a data type.	Subtracting 1 from 0 in an unsigned integer, causing it to wrap around to the maximum value (e.g., 255).

5.3 Logic error

A logic error is a mistake made in the design of a program, such as using an inappropriate mathematical formula or control structure. It is unlikely to generate an error message. The program may run but will not produce the expected results.

While runtime errors typically cause the program to crash or fail immediately, logic errors may go unnoticed as they don't necessarily disrupt the program's execution but instead cause incorrect results. The only way to detect the logic error is by running a test case and observing that the actual output of the program is different from the expected output.

5.4 Designing test cases

The basic objective of writing test cases is to validate the testing coverage of the application. A test case consists of components such as input, purpose, and expected output, which help determine if a feature of the application works as intended.

A test case is designed in such a way that only one thing is tested at a time (i.e.: atomic test case). Do not overlap or complicate test cases. To detect runtime errors and logic errors, it is important to run the programs through multiple test cases. A test case should aim to cover a wide range of data, including normal, abnormal, extreme, and boundary data, ensuring that all edge cases are tested.

1. Normal data

Normal data is test data that is sensible, expected, and should be accepted and processed by the system. In general, under normal conditions, at least one test case should be designed for each type of expected input.

2. Abnormal data

Abnormal data (also known as invalid or erroneous test data) falls outside of what is acceptable and should be rejected by the system.

3. Extreme data

Extreme data is at the upper or lower limits of expectations that should be accepted by the system. Or where special handling of the input data is required (e.g.: use of -1 as NULL value).

4. Boundary data

Boundary data *can be extreme data* and also includes the immediate values before or beyond the limits of expectations that should be rejected.

For example, in a system that was designed to accept and process test marks (in percentages):

Input (test cases)	Purpose (test for ...)	Expected output
10, 25, 65, 88	Normal data	Accepted
-1, 101, 200, -55	Abnormal data	Rejected
0, 100	Extreme (or Boundary) data	Accepted
-1, 101	Boundary data	Rejected

In systems that deal with text, the extreme values are defined by how long the text can be.

Input (test cases)	Purpose (test for ...)	Expected output
"" (empty string)	Extreme data	Accepted
"ABCDEF..." (maximum length)	Extreme data	Accepted

5.5 Designing test plan

A test plan is a comprehensive set of test cases that covers all types of data and scenarios. It includes expected outputs for each test case. When testing the program, a fourth column is added to record the actual output produced by the program. The actual output is then compared to the expected output to ensure the program functions as intended.

Input (examples)	Purpose	Expected output	Actual output
10, 25, 65, 88	Normal data	Accepted	(record exactly what was output by the program, for example a screenshot of a sample run of the program)
-1, 101, 200, -55	Abnormal data	Rejected	
0, 100	Extreme data	Accepted	
-1, 101	Boundary data	Rejected	

6. RECURSION

Recursion is the process by which a function calls itself as part of its execution. Each recursive function has two key components: a stopping condition (or base case) and a recursive call. The stopping condition prevents infinite recursion by ensuring that the function eventually stops calling itself and begins to “unwind.”

Features of a recursive function:

- It must be defined in terms of itself.
- A stopping condition (or base case) must be included to prevent infinite recursion, allowing the function to return a result.
- The function may have multiple stopping conditions to handle different cases.
- For input values other than the stopping condition, the function calls itself.
- The stopping condition must be reached after a finite number of calls.

6.1 Comparing recursion and iteration

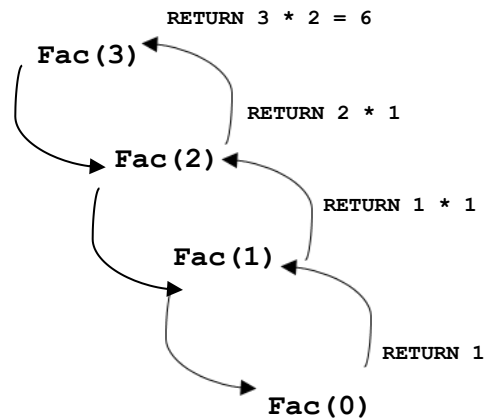
Feature	Recursion	Iteration
Simplicity	More elegant for problems that naturally break into smaller subproblems (e.g., tree traversal).	Can be more complex for problems that involve recursive patterns.
Memory Usage	Can use a large amount of memory due to call stack growth, especially for deep recursion.	Fixed memory usage; does not use extra memory for function calls.
Execution Speed	May be slower due to overhead from function calls and maintaining the call stack.	Generally faster as it avoids the overhead of recursive function calls.
Risk of Errors	Risk of stack overflow if base case is missing or if recursion depth is too large.	No risk of stack overflow; risks are usually related to incorrect loop conditions or logic errors.
Ease of Debugging	More challenging to trace due to repeated function calls and deeper call stack.	Easier to debug since it involves straightforward loop constructs.
Proof of Correctness	Easier to prove mathematically using induction, especially when the problem follows a recursive structure.	Harder to prove, but still possible with loop invariants or inductive reasoning.
Code Size	Typically shorter and more elegant, especially for naturally recursive problems.	May require more lines of code, especially for problems that can be described recursively.
Use Cases	Best for problems that can be divided into smaller, similar subproblems (e.g., factorial, Fibonacci, binary search).	Suitable for repetitive tasks that don't require breaking down problems into smaller subproblems (e.g., summing numbers in a list, printing values).
Time Complexity	May be less efficient due to overhead from multiple function calls.	Generally more time-efficient for simple problems as there's no overhead from function calls.

Example 1: Define the calculation of a factorial, $\text{Fac}(N)$ as a recursive routine.

$0! = 1$
 $1! = 1$
 $2! = 2 \times 1 = 2$
 $3! = 3 \times (2 \times 1) = 3 \times 2! = 6$
 $4! = 4 \times (3 \times 2 \times 1) = 4 \times 3! = 24$

 $n! = n \times [(n-1) \times (n-2) \times \dots \times 2 \times 1]$
 $= n \times (n-1)!$

A trace tree diagram for the recursive function call $\text{Fac}(3)$ is as follows:



Recursive function $\text{Fac}(N)$ in pseudocode:

```

FUNCTION Fac(N: INTEGER) RETURNS INTEGER
    IF N = 0 THEN
        RETURN 1
    ELSE
        RETURN N * Fac(N-1)
    ENDIF
ENDFUNCTION

```

Nothing is returned until the function stops calling itself. As soon as the stopping condition is reached, in this case when $N = 0$, the `RETURN 1` statement at the end of the function is reached and control is passed back to the next statement after the last call to `Fac` which is `ENDIF`. This process of returning from each recursive call is called *unwinding*.

Example 2: Consider the Fibonacci sequence. The first two terms are 0 and 1, and subsequent terms are the sum of the previous two terms. This can be expressed as:

$$\text{Fibonacci}(n) = \begin{cases} 0, & \text{if } n = 0 \\ 1, & \text{if } n = 1 \\ \text{Fibonacci}(n-1) + \text{Fibonacci}(n-2), & \text{if } n > 1 \end{cases}$$

The recursive pseudocode for $\text{Fibonacci}(n)$ to output the n^{th} term is:

```

FUNCTION Fibonacci(n: INTEGER) RETURNS INTEGER
    IF n = 0 THEN
        RETURN 0
    ELSE IF n = 1
        RETURN 1
    ELSE
        RETURN Fibonacci(n-1) + Fibonacci(n-2)
    ENDIF
ENDFUNCTION

```

And the iterative pseudocode is:

```
FUNCTION Fibonacci(n: INTEGER) RETURNS INTEGER
  IF n = 0 THEN
    RETURN 0
  ELSE IF n = 1
    RETURN 1
  ENDIF
  num1 ← 0
  num2 ← 1
  FOR i ← 2 TO n
    num3 ← num1 + num2
    num1 ← num2
    num2 ← num3
  ENDFOR
  RETURN num3
ENDFUNCTION
```

Look at both the recursive and iterative algorithms. The recursive algorithm follows the mathematical definition closely, making it simpler and shorter than the iterative version.

6.2 Use of Call Stack

When a function is called, memory is allocated for its execution. The **call stack** stores information about active function calls during the program's runtime. It is composed of **stack frames**, where each frame corresponds to a function call that has not yet completed.

Each stack frame stores:

- The return address: The instruction to return to when the function finishes.
- Parameters: The input values passed to the function.
- Local variables: Variables created within the function.

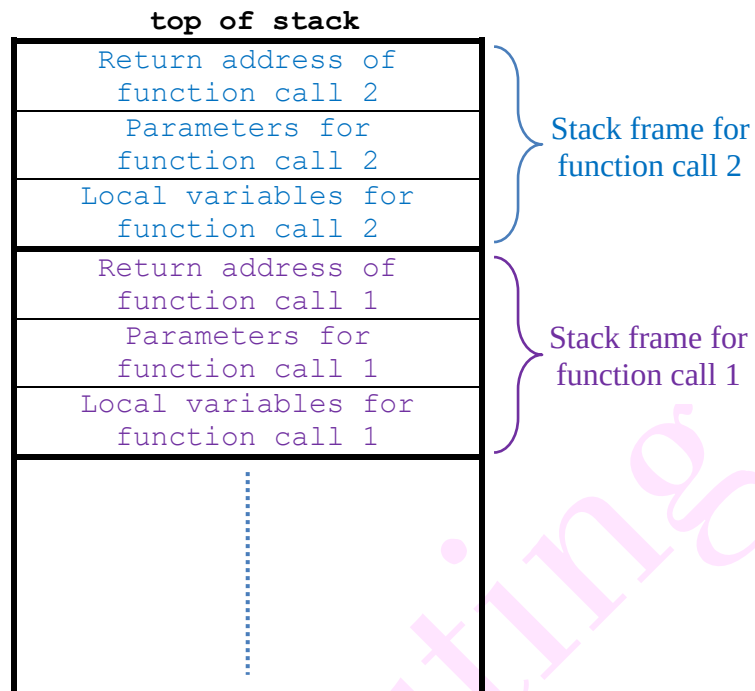
When a function calls another function (or itself in the case of recursion), a **new stack frame** is pushed onto the top of the stack. In recursion, each new function call adds another frame with a return address, parameters, and local variables. Once a function finishes executing, its stack frame is **popped** from the stack, and control returns to the stored return address.

If recursion continues indefinitely without reaching a **base case**, the stack will eventually run out of space. The call stack has a maximum size, so too many recursive calls will cause a **stack overflow**, leading to a program crash.

For example, in a recursive factorial function $\text{Fac}(3)$, the call stack works as follows:

1. $\text{Fac}(3)$ is called → pushes a stack frame.
2. $\text{Fac}(2)$ is called inside $\text{Fac}(3)$ → pushes another frame.
3. $\text{Fac}(1)$ is called inside $\text{Fac}(2)$ → pushes another frame.
4. $\text{Fac}(0)$ is called inside $\text{Fac}(1)$ → reaches the base case, returns 1, and stack starts unwinding.
5. Each function returns, and its stack frame is popped until $\text{Fac}(3)$ finally returns 6.

If we forgot the base case, the function would keep calling itself indefinitely, causing a **stack overflow**.



7. VIDEO REFERENCES

7.1 Trace Tables Explained (Algorithm / Flowchart Dry Runs):

- Author: GCSE GURU
- Upload Date: Jan 11, 2021
- URL: <https://www.youtube.com/watch?v=jqegERorXPI>

7.2 Validation and Verification Explained:

- Author: GCSE GURU
- Upload Date: Jan 17, 2021
- URL: <https://www.youtube.com/watch?v=Nmwtf0fAUfI>

7.3 Syntax, Runtime, and Logical Errors in Python:

- Author: GCSE GURU
- Upload Date: Nov 9, 2018
- URL: <https://www.youtube.com/watch?v=ToPP5UGgJUM>

7.4 Test Data Explained

- Author: GCSE GURU
- Upload Date: Jan 17, 2021
- URL: <https://www.youtube.com/watch?v=tWwcefljMAQ>

7.5 Recursion – CS50 Shorts:

- Author: CS50
- Upload Date: Oct 25, 2017
- URL: <https://www.youtube.com/watch?v=mz6tAJMvmfM>

7.6 The Call Stack:

- Author: Computer Science Lessons
- Upload Date: Oct 16, 2014
- URL: <https://www.youtube.com/watch?v=Q2sFmqvpBe0>